

CloudPulse – Demo Script & Viva Questions

Oct 3, 2026 · @Cecil

A run order for the live demo and the questions examiners usually ask, with short answers to practise.

Before the demo (10 minutes early)

- ☐ Laptop charged; close Chrome tabs you don't need (your PC's memory alert is real and fires above 88%).
- ☐ MySQL running: PowerShell `Get-Service MySQL80` shows Running.
- ☐ IntelliJ: ▶ Run CloudMonitorApplication; log shows "Email alerts: ready" and "Started CloudMonitorApplication".
- ☐ PowerShell in the `frontend` folder: `npm run dev`.
- ☐ Chrome: open localhost:5173 and log in as **Admin**; theme set to dark.
- ☐ Alerts page: **Acknowledge all**, so the list starts clean.
- ☐ Task Manager open (Performance tab) for the real-metrics comparison.
- ☐ Gmail open in a second tab.
- ☐ A recent PDF report downloaded, as a backup if export is slow.
- ☐ Phone hotspot ready in case the college Wi-Fi blocks Gmail.

Demo script (about 8 minutes)

Time	Show	Say
0:00	Login page, click Admin , Sign in	"Login uses JWT tokens; passwords are stored as BCrypt hashes."
0:30	Dashboard: point at the green Live badge and the clock	"Every 3 seconds the server pushes new data over WebSocket. I am not refreshing the page."
1:30	Switch the utilisation chart to 24h	"History is stored in MySQL; the chart averages it into at most 90 points."
2:00	Resources → CECIL (this PC) , Task Manager beside it	"This one is real, read by OSHI. Memory and uptime match Task Manager."
3:00	Resources → a cloud server → Stop , then Start	"Only admins can do this; it is checked on the server."

Time	Show	Say
3:45	Alerts → Rules , open "CPU critical"	"Rules are stored in MySQL; I can change the threshold here."
4:15	Alerts → Email → Send test email , show Gmail	"Critical alerts are emailed. The Gmail App password is in environment variables, not in the code."
5:15	Costs : budget bar, ways to save	"Burn rate times 730 hours gives the monthly estimate. The tip comes from 24 hours of usage history."
6:15	Logs → Errors → search "timed"	"Live log stream with filters; lines get worse when a server is under load."
7:00	Export → Full report , open the PDF	"The PDF is built in Java with OpenPDF."
7:30	Log out, log in as viewer	"A viewer sees everything but the admin buttons are gone."

If a real alert pop-up appears during the demo, stop and point at it: it shows the live system working.

If something goes wrong

Symptom	Quick fix
Top bar shows "Reconnecting"	Backend is not running: ► Run in IntelliJ, wait for "Started".
Backend fails with "Communications link failure"	MySQL stopped: PowerShell as admin, <code>net start MySQL80</code> .
"Port 8080 already in use"	Another copy is running: stop it in IntelliJ (■), or restart IntelliJ.
Page blank or <code>npm run dev</code> closed	Run <code>npm run dev</code> again in the frontend folder.
Test email fails	Check internet; show the email already received earlier as proof.
Too many alerts on screen	Alerts → Acknowledge all.
Anything else	Stay calm, explain from the slides and architecture diagram, then retry once.

Viva questions and answers

Keep each answer to two or three sentences, then stop. If you don't know, say what you do know and how you would find out.

Project basics

- 1. What does your project do?** It monitors cloud servers live: CPU, memory, disk and network every 3 seconds, with alerts, email, cost tracking, logs and reports, for Admin and Viewer users.
- 2. Is the cloud data real?** Ten cloud servers are simulated because real AWS or Azure accounts cost money; the eleventh resource, my own PC, is real and read through OSHI. The simulation sits behind one service, so real APIs could replace it.
- 3. How is the simulation realistic?** Each server has a base load by type, a daily curve that peaks in the afternoon, random noise, and a 0.15% chance per tick of a 30 to 90 second spike.
- 4. Who would use this?** An operations or DevOps team, or a small company that runs servers on more than one cloud.

5. **How is it different from AWS CloudWatch?** CloudWatch covers only AWS and is complex; CloudPulse shows several providers on one simple screen and adds cost tips. It is a learning project, not a replacement.

Java and Spring Boot

6. **Why Spring Boot?** It sets up a web server, REST API, database access, scheduling and security with very little configuration.
7. **What is a REST API?** A set of URLs the browser calls with HTTP methods (GET, POST, PUT, DELETE) that return JSON, for example GET /api/resources.
8. **How does the 3-second update work?** `MonitoringService.collect()` has `@Scheduled(fixedRate = 3000)`; Spring calls it every 3 seconds.
9. **What is dependency injection?** Spring creates objects (beans) and passes them into constructors, so `MonitoringService` receives `AlertService` instead of creating it.
10. **What is a DTO?** A Data Transfer Object: a simple class (a Java record here) that shapes the data sent to the browser, like `CostReport`.
11. **Controller vs service vs repository?** The controller receives the HTTP request, the service holds the logic, and the repository talks to the database.
12. **Why send emails on another thread?** Gmail can take seconds; a background thread keeps the 3-second loop on time.

Real-time and frontend

13. **What is WebSocket and why use it?** A connection that stays open so the server can push data at any time. Without it the browser would have to ask every few seconds (polling), which is slower and wasteful.
14. **What is STOMP?** A simple messaging protocol on top of WebSocket with topics; the browser subscribes to /topic/live, /topic/alerts and /topic/logs.
15. **Why React and not JavaFX or JSP?** React gives smooth live charts and works in any browser and on phones; the business logic is still all Java.
16. **What is a component and what is state?** A component is a reusable piece of UI (like a Gauge); state is data that, when it changes, makes React redraw that part.

Database

17. **Why MySQL instead of H2?** H2 in memory loses everything on restart; MySQL keeps data permanently. H2 was used first for quick testing.
18. **What is JPA / Hibernate?** It maps Java classes to tables, so I save objects instead of writing SQL. With `ddl-auto=update` it creates the tables.

19. **What are your tables and relations?** Six tables; cloud_resources is central: one resource has many metric_snapshots, alert_rules and alerts; one rule has many alerts.
20. **How do you stop the history table growing forever?** A job deletes rows older than 24 hours every 10 minutes, and only one point per 15 seconds is saved.
21. **What was the "value" bug?** value is a reserved word in SQL, so creating the alerts table failed; I renamed the column to metric_value with @Column.

Security

22. **How does login work?** The server checks the password against the BCrypt hash and returns a signed JWT; the browser sends it in the Authorization header on every request.
23. **What is a JWT?** A signed token holding the username, role and expiry; the server can verify it without storing sessions.
24. **Why BCrypt?** It is a slow, salted hash, so stolen hashes are very hard to crack.
25. **Can a viewer stop a server by calling the API directly?** No.
@PreAuthorize("hasRole('ADMIN')") is checked on the server and returns 403.
26. **Where is the Gmail password stored?** Only in environment variables of the run configuration, never in the code or on GitHub.
27. **Any weak points?** The MySQL password and JWT secret are in application.properties; fine for a demo, but in production they should be environment variables.

Features and logic

28. **How is monthly cost calculated?** Sum of the hourly price of running resources times 730 hours ($365 \times 24 \div 12$).
29. **How do saving tips work?** A resource that is 40% or more of the bill gets "run only in working hours" (saves half); a server averaging under 25% CPU over 24 hours gets "downsize".
30. **How do you avoid alert spam?** A 2-minute cooldown per rule and server, and at most 20 emails per hour.
31. **How do you read real CPU usage?** OSHI compares CPU tick counters between two readings 3 seconds apart.
32. **Why did disk show 33% but Task Manager 2%?** Different things: CloudPulse shows space used, Task Manager shows disk activity. I relabelled it "Disk used".

Testing and future

33. **How did you test?** 16 manual test cases covering login, roles, live data, alerts, email, costs, PDF, logs, persistence and mobile; all pass.

34. **What was hardest?** Making live updates reliable and debugging setup issues, such as the reserved column name and the email environment variable with a stray space.
35. **What would you add next?** Real AWS/Azure APIs, logs stored in a database, prediction like "disk full in 3 days", JUnit tests and Docker deployment.
36. **What did you learn?** Building a full-stack app end to end: REST, WebSocket, database design, security, background jobs and a React UI, plus Git for version control.